

Stuart School of Business



ILLINOIS INSTITUTE OF TECHNOLOGY

Project Report

The People Analytics at Teach for America (A)

Predictive Analytics

Dr. Dinakar Jayarajan

Submitted by - Group 4

Ishitha Nanda Kumar

INDEX

1. Dataset Overview	3
3. Final Dataset Structure	5
Implementation	6
Accuracy	6
Sensitivity : Indicates how well the model identifies applicants who will not complete the admissions process.	
A sensitivity of 7.58% is extremely low, demonstrating poor performance in identifying class 0 cases. This means the model is missing a majority of applicants in this category.	6
Specificity: Measures how well the model predicts completed admissions (class 1).	
With a specificity of 96.02%, the model performs very well in identifying applicants who are likely to complete the admissions process.	6
Kappa Statistic : This metric measures the agreement between the predicted and actual classes, beyond random chance.	
Kappa = 0.0507, indicating very low agreement and limited ability of the model to capture meaningful patterns, especially for class 0.	6
Business Insights	7
Key Metrics:	13
Accuracy:	13
Sensitivity:	14
Specificity:	14
Kappa Statistic:	14
Implementation	18
Key Metrics:	18
Confusion Matrix	18
Key Findings:	19
Business Insights	20

INTRODUCTION

Teach For America (TFA), a national nonprofit founded in 1989, recruits high-performing graduates to teach in underprivileged schools across the United States for two years, with the dual mission of addressing educational inequity and developing transformational leaders.

As application rates fell by 16% for the third consecutive year, TFA grappled with a competitive recruitment environment and inefficiencies in its processes. However, after peaking at about 57,000 in 2013, applications declined by 12% in 2014, 10% in 2015, and 16% in 2016. To reverse this trend, TFA is using data-driven strategies and people analytics to enhance recruitment efficiency, improve candidate engagement, and optimize resource allocation.

CHALLENGES

The key challenges highlighted in the "People Analytics at Teach For America (A)" case include:

1. **Declining Applications:** Over recent years, TFA experienced a significant and sustained drop in applications, with a 16% decline in 2016 compared to the previous year. This trend threatens their ability to recruit high-quality corps members and sustain their impact.
2. **Misaligned Recruitment Timing:** TFA's recruitment timeline starts during candidates' senior year, which often clashes with when top graduates have already accepted other job offers, reducing the pool of eligible and interested applicants.
3. **Leaky Recruitment Funnel:** There were notable inefficiencies in the recruitment process. Potential applicants dropped out at various stages — failing to complete applications, declining offers, or deciding not to matriculate even after accepting offers.

DATA SET OVERVIEW

1. Dataset Overview

The dataset contains information related to applicants, including their academic performance, essay characteristics, and application timelines. Upon initial exploration, the following observations were made:

- **Structure of the Data:**

The dataset includes both numerical and categorical variables, as well as date-based columns. The dataset has 31246 observations and 24 variables. Some key features are:

- Academic Details: Gpa, stem (STEM background indicator), and schoolsel (school selectivity).
- Essay Characteristics: essay1length, essay2length, essay3length, essayuniquewords, and essayssentiment.
- Application Timelines: signupdate, starteddate, submitteddate.
- Categorical Attributes: Variables such as appdeadline and completedadm.

- **Data Quality Issues:**

- Missing values were found in critical variables such as schoolsel and essay1length.
- Numerical columns had diverse ranges, requiring standardization.
- Some columns were redundant due to high correlations or minimal relevance.

- **Initial Observations:**

The dataset required significant preprocessing to handle missing data, scale numerical variables, and reduce dimensionality for analysis.

2. Data Cleaning and Preprocessing

To ensure the dataset was ready for analysis and modeling, the following steps were undertaken:

a. Handling Missing Values

- **schoolsel:** Missing values were replaced with zero
- **essay1length:** Missing values were imputed with zero

b. Normalization of Numerical Variables

To standardize numerical columns and eliminate scale differences, we applied normalization using feature scaling. The following variables were normalized:

- gpa

- essayuniquewords
- essayssentiment
- signupdate, starteddate, and submitteddate

This step ensured uniformity across all numerical features, improving the compatibility of the data for analysis.

c. Dimensionality Reduction

- **Correlation Analysis:** Conducted pairwise correlation among numerical variables. Variables with a correlation coefficient above 0.85 were removed to address multicollinearity.
 - Removed columns: essay1length, essay2length, essay3length.

d. Removal of Irrelevant Columns

Columns with limited relevance or excessive categories were excluded to streamline the dataset:

- **Identifying Variables:** personid, appyear, and schoolsel_chr.
- **Grouping Details:** Columns such as major1, major2, minor, major1group, major2group, and minorgroup.
- **Institutional Information:** undergrad_uni.

e. Factor Conversion

Several categorical variables were converted into factors to align with analysis requirements:

- stem, schoolsel, appdeadline, and completedadm.

3. Final Dataset Structure

After cleaning and preprocessing, the refined dataset contains the following key features:

- **Academic Performance:**
 - gpa: Applicant's Grade Point Average.
 - stem: Indicator of a STEM background (categorical).
 - schoolsel: School selectivity level (categorical).
- **Essay Metrics:**
 - essayuniquewords: Number of unique words in all essays.
 - essayssentiment: Sentiment score of essays.
- **Application Timeline:**

- signupdate: Date the application was signed (normalized).
- starteddate: Date the application process started (normalized).
- submitteddate: Date the application was submitted (normalized).
- **Additional Information:**
 - attendedevent: Indicator for attendance at application-related events.
 - completedadm: outcome variable; 1 = completed all steps of admissions process; 0 = withdrew at some point during admissions process

This refined dataset, free from missing values and redundant features, is well-prepared for further exploration and predictive modeling.

CLASSIFICATION MODELS

KNN:

KNN is a distance-based classification algorithm that predicts an applicant's class by identifying the nearest data points in the feature space. KNN performs well with balanced datasets but can struggle with imbalanced distributions and overlapping feature spaces.

Implementation

Objective: Provide recruiters with a model that evaluates applicants based on similarity in features, predicting their likelihood of completing the admissions process.

Key Metrics

Accuracy: Represents the overall percentage of correct predictions. An accuracy of 78.76% is reasonable; however, it is heavily influenced by the dominant class (completed admissions). This skews the performance and reduces the reliability for predicting minority outcomes (class 0).

Sensitivity : Indicates how well the model identifies applicants who will not complete the admissions process.

A sensitivity of 7.91% is extremely low, demonstrating poor performance in identifying class 0 cases. This means the model is missing a majority of applicants in this category.

Specificity:

- Measures how well the model predicts completed admissions (class 1).
- With a specificity of 96.02%, the model performs very well in identifying applicants who are likely to complete the admissions process.

Kappa Statistic :

- This metric measures the agreement between the predicted and actual classes, beyond random chance.
- Kappa = 0.0591, indicating very low agreement and limited ability of the model to capture meaningful patterns, especially for class 0.

Business Insights

Strengths: KNN is highly effective at identifying applicants who will complete the admissions process, making it suitable for scenarios where minimizing false positives for class 1 is critical.

Weaknesses: The model's poor sensitivity and low kappa indicate it is not suitable for reliably identifying applicants unlikely to complete the process.

Recommendation: While KNN is useful for identifying clear positives (class 1), its low performance on class 0 and inability to generalize effectively suggest exploring alternative models such as Random Forest or Logistic Regression for a more balanced and insightful analysis.

NAIVE BAYES:

Naive Bayes is a probabilistic classifier that predicts the class of completion by an applicant through independent contributions from the features. Naive Bayes does well with categorical data, but it fails on highly correlated variables.

Implementation

Objective: Give recruiters a model which can quickly and easily understand and which screens the applicants based on their likelihood of completing.

Accuracy: Represents the overall percentage of correct predictions. Although this seems quite reasonable, it is being skewed by a very large number of class 1 cases (completed admissions).

Sensitivity (Recall for Class 0): Gives the model's performance in terms of correctly predicting the class 0-not completing the admission.

Low sensitivity 6.29% indicates poor performance of the model in correctly predicting applicants who will not complete the admission process.

Specificity: Indicates how well the model identifies class 1, which are completed admissions.

A high specificity, 96.6%, shows very good performance in predicting the applicants who will complete the admission.

Kappa Statistic: This metric measures the agreement between the predicted and actual classes beyond random chance.

Kappa = 0.0415 very low agreement; this indicates the limited capacity of the model to grasp any meaningful patterns in Class 0.

Business Insights

- Naive Bayes is best deployed as a baseline model for quick decision-making.
- Useful for triaging applicants where data completeness and simplicity are priorities.

SUPPORT VECTOR MACHINES (SVM):

SVM finds an optimal hyperplane that separates classes (for example, completedadm = 0 or 1). It was tested in two flavors:

- Linear Kernel: Appropriate for data that is straightforward and linearly separable.
- RBF Kernel: Perceives the nonlinear relationships among predictors.

Implementation on TFA Context

Objective: Give recruiters high-accuracy predictions for effective targeting on high-risk candidates.

Linear Kernel:

Key Metrics

Accuracy: Here, the accuracy of the model is 80.17%, majorly because of the correct prediction of class 1, which referred to completed admissions. However, the value is misleading as all class 0 cases were missed by the model.

Sensitivity :Sensitivity refers to the model's ability in correctly identifying actual class 0 cases where a student did not complete the admission process.

A sensitivity of 0 means that no class 0 cases were predicted by the model; it completely missed applicants who did not complete the admissions process.

Specificity: Specificity measures the model's accuracy in identifying true class 1 cases (completed the admissions).

In this case, the specificity is 100%, with the model correctly predicting all class 1 cases and failing to misclassify any class 1 cases into class 0.

Kappa Statistic: Kappa considers the agreement between the predicted and actual values beyond random chance.

A Kappa of 0 indicates no agreement beyond chance, mainly because the model completely failed on class 0 predictions.

The model is effectively biased toward the majority class, class 1, which skews performance.

RBF Kernel:

Key Metrics

Accuracy: Similar to the linear kernel, the accuracy is 80.28%, but again, this result is dominated by the model capability of predicting class 1 - completed admissions.

Sensitivity: Sensitivity increases a bit compared to the linear kernel and stays low - 0.56%, which again means the model isn't able to predict all class 0 cases reliably.

Specificity: The RBF kernel also runs at 100% specificity, perfectly identifying class 1 cases without errors.

Kappa Statistic: The Kappa statistic of 0.009 indicates very little agreements between classes beyond that expected by chance.

This small value will now reflect that the model is not properly balanced in its prediction among the classes.

Business Insights

- The RBF kernel SVM is ideal in scenarios where precision and recall are vital, such as selecting high-potential applicants.
- SVM results can be fed back into TFA's CRM to provide actionable recruiter guidance, including prioritizing outreach to high-risk applicants.

Decision Tree:

A decision tree operates by systematically splitting a dataset into branches based on specific decision rules derived from the input features, enabling it to predict outcomes efficiently.

Implementation Context:

Goal: To provide accurate predictions for decision-making by identifying both high-risk cases (class 0) and low-risk cases (class 1) effectively.

Key Metrics:

Accuracy: The accuracy of the model is **80.08%**, indicating that most predictions were correct. While the accuracy seems high, it should be taken in mind that the model performs well for class 1 but poorly for class 0.

Sensitivity: Sensitivity measures the model's ability to correctly identify actual class 0 cases (those who did not complete the process). Sensitivity here is about 2.9%, which is low. This indicates that the model almost completely fails to identify class 0 cases. Missing these cases means it is ineffective in addressing the high-risk group, which is critical for decision-making.

Specificity: Specificity measures the model's accuracy in identifying true class 1 cases (those who completed the process). Specificity is **99.16%**, which is very high. The model accurately identifies class 1 cases, suggesting a strong bias toward the majority class. While this may be helpful in predicting completed cases, it overlooks the minority class, rendering the model ineffective for comprehensive predictions.

Kappa Statistic: Kappa considers the agreement between predicted and actual values beyond random chance. Kappa is **0.0319**, which indicates very low agreement. This low value confirms that the model performs poorly in predicting the minority class (class 0), leading to minimal agreement beyond random chance. The model's performance is imbalanced and heavily skewed toward the majority class.

The decision tree starts with the root node, which typically represents the most important feature for classification or prediction. In this case, the model prioritizes gpa as the first decision criterion. Each internal node makes decisions based on a feature and a threshold, and certain features appear multiple times across different branches, indicating their high importance in the model's decision-making process. The number of splits from the root to the leaf nodes reflects the model's complexity. From the diagram, the tree shows a moderate depth, suggesting that it captures meaningful patterns in the data without being overly complex.

To improve the model's prediction rate for class 0, weights of 10 and 1 were assigned to class 0 and class 1, respectively. A Random Forest model was then built using these weights. Random Forest builds multiple decision trees during training and combines their outputs using majority voting for classification or averaging for regression to make accurate and robust predictions. Random Forest reduces overfitting by averaging the predictions of multiple trees, making it more accurate and stable.

Random Forest:

The accuracy of the Random Forest model slightly decreased from 80% to 79.95%. However, its sensitivity improved significantly from 2.9% to 5.3%, indicating that the model's ability to correctly predict class 0 cases has increased. On the other hand, specificity dropped from 99.16% to 98.4%, meaning the accuracy of predictions for class 1 decreased slightly. Similarly, the Kappa value also declined. While the Random Forest model lost some overall accuracy due to the effect of weights, it achieved a notable improvement in identifying class 0 cases, which addresses the imbalance issue more effectively.

The chart shows the features that are most significant in the predictions made by the Random Forest model. Among them, essayuniqueword, gpa, and essaysentiment rank as the top three in importance.

Business Insights:

Decision Tree:

Strengths: Decision Tree is easy to interpret and provides clear rules for identifying applicants likely to complete the admissions process.

Weaknesses: Its moderate sensitivity and low kappa indicate challenges in accurately identifying applicants unlikely to complete the process. It is also prone to overfitting.

Recommendation: While useful for interpretability, exploring more robust models like Random Forest or Gradient Boosting is recommended for improved performance and generalization.

Random Forest :

Strengths: Random Forest is robust and handles class imbalances well, providing high accuracy and generalization across both classes.

Weaknesses: Despite its strong performance, the model lacks interpretability and may require more computational resources compared to simpler models.

Recommendation: Random Forest is suitable for scenarios requiring high accuracy and balanced performance, but for greater interpretability, consider simpler models like Logistic Regression or Decision Tree when appropriate.

Artificial Neural Networks (ANN):

Artificial Neural Networks (ANNs) are excellent tools capable of capturing complex, non-linear relationships in data. They excel in both classification and prediction tasks. Although ANNs are normally more effective with multi-class datasets, we decided to apply the model to our binary dataset to explore its potential for predicting application outcomes. The features used in this model were chosen based on their significance, as identified by the Random Forest feature importance plot.

Implementation

Objective: Provide recruiters with a predictive model that assesses the probability of candidates completing the admissions process based on key application indicators such as GPA, essay metrics, and deadlines.

Key Metrics:

Accuracy : The model achieved an accuracy of 80.17%, indicating adequate performance in predicting application results. However, this metric is skewed by the dominant class (completed applications), reducing its reliability for identifying non-completions (class 0).

Sensitivity : The model's ability to accurately identify candidates who successfully complete their applications (class 1). This model showed 100% sensitivity, correctly identifying all candidates who finished the application process. This ensures that no completions are missed.

Specificity : Referring to the model's ability to correctly recognize incomplete applications (class 0). The model's specificity is 0%, which means it failed to properly forecast any applicants who did not complete their applications. This is a significant restriction in making balanced predictions.

Balanced Accuracy : The average of sensitivity and specificity after accounting for performance in both classes. With a balanced accuracy of 50%, the model struggles to generalize between the two classes, emphasizing its failure to reliably predict non-completions.

Confusion Matrix

- **True Positives (TP)**: 5010 (Correctly predicted as completed).
- **False Positives (FP)**: 1239 (Incorrectly predicted as completed but did not complete).
- **True Negatives (TN)**: 0 (Correctly predicted as not completed).
- **False Negatives (FN)**: 0 (Incorrectly predicted as not completed).

Key Findings:

1. **High Likelihood of Completion**: According to the model, the majority of candidates have a high probability of finishing their applications, often exceeding 80%.

For example:

Candidate 1 has an 86.12% chance of completion.

Candidate 214 has a 91.49% chance of completing.

Candidate 484 has a 92.06% chance of completing.

These findings show that the model's variables, such as GPA and essay metrics, are strong predictors of application completion.

2. **Statistical Summary of Completion Probabilities**: Minimum Probability: ~63% (candidate with the lowest likelihood).

Maximum Probability: ~94% (candidate with the highest likelihood).

Average Probability: The average likelihood of application completion is around 80%, highlighting a general trend of candidates being likely to complete the application process.

Analysis of ANN Model Plot

The ANN model consists of three layers: an input layer with eight variables (such as gpa, essayuniquewords, and signupdate), a hidden layer with one node to capture non-linear relationships, and an output layer that predicts probability for two classes. completedadm_0 (not completed), completedadm_1 (completed)

The ANN plot shows the weights that connect each input feature to the hidden node and the hidden node to the output nodes. These weights determine how each feature influences the predictions. For Instance:

- essayuniquewords has a significant weight (0.055), indicating the magnitude of the essay vocabulary impact on positive completions.
- gpa also has a relatively high positive weight (0.32), indicating that higher GPAs strongly contribute to the likelihood of completing the application.
- Schoolsel_2 and schoolsel_3 on the other hand have negative weights, indicating that they reduce the likelihood of completion.
- Bias Weights: The intercept terms demonstrate how the network adjusts predictions without relying simply on inputs.

Business Insights

Strengths:

- **Ability to Capture Complex Relationships and High Predictive Accuracy for Class 1:** The ANN model successfully identifies non-linear relationships between variables such as GPA, essay sentiment, and application deadlines, resulting in accurate predictions for class 1 (completed), making it an effective tool for finding high-potential applicants.

Weaknesses:

- **Poor Performance on Class 0:** The model struggles to forecast candidates who are unlikely to complete their applications, restricting its ability to provide a balanced and accurate assessment.
- **Imbalanced Dataset Impact:** The large proportion of completed applications in the dataset skews the model's performance, reducing its ability to be generalized.

Recommendations:

- **Targeted Support:** We can use the ANN Model's findings to identify candidates with lower completion probability for targeted interventions. For Example :

Candidate 237 has a 63.96% chance of completing.

Candidate 252 has a 65.17% chance of completing.

Candidate 368 has a 65.49% chance of completing.

Recruiters could offer further assistance, such as tailored follow-ups or application assistance, to help these prospects succeed and complete the admission.

MODEL RECOMMENDATION

Model	Accuracy	Sensitivity	Specificity	Kappa Statistic
KNN	78.76%	7.91%	96.02%	0.0591
Naive Bayes	78.70%	6.29%	96.60%	0.0415
SVM (Linear Kernel)	80.17%	0%	100%	0
SVM (RBF Kernel)	80.28%	0.56%	100%	0.009
Decision Tree	80.08%	2.90%	99.16%	0.0319
Random Forest	79.95%	5.30%	98.40%	0.04
Artificial Neural Network (ANN)	80.17%	100%	0%	0

Model Recommendation: Decision Tree

Based on the analysis of various predictive models, the decision tree emerges as the best option for the Teach For America (TFA) case. This recommendation is grounded in the following observations:

1. **Accuracy:** The decision tree achieved an accuracy of 80.08%, demonstrating its reliability in correctly predicting outcomes for most applicants.
2. **Performance Across Classes:** Although the sensitivity for predicting Class 0 (non-completion) remains low, the decision tree showed the best ability to balance both specificity and prediction for Class 1 (completion) relative to other models.
3. **Interpretable Results:** Decision trees offer a clear and interpretable structure, allowing recruiters to easily understand the factors influencing predictions, such as GPA and essayuniquewords.
4. **Actionable Insights:** The decision tree's output provides practical insights into key predictors, enabling targeted interventions to address applicant dropout at specific stages of the process.

APPENDIX

```
# Load required libraries
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
library(caret)
```

```
library(e1071)
```

```
library(C50)
```

```
library(neuralnet)
```

```
library(fastDummies)
```

```
library(kernlab)
```

```
library(ggdendro)
```

```
library(tidyverse)
```

```
library(class)
```

```
library(randomForest)
```

```
#Load and Inspect the Dataset
```

```
people <- read.csv("People.csv") str(people)
```

```
summary(people)
```

```
#Data Cleaning
```

```
# Fill missing values in 'schoolsel' and 'essay1length' with 0
```

```
pa.df <- people %>%
```

```
  mutate(schoolsel = ifelse(is.na(schoolsel), 0, schoolsel)) %>%
```

```
  mutate(essay1length = ifelse(is.na(essay1length), 0, essay1length))
```

```
# Remove irrelevant columns
```

```
pa.df <- pa.df %>%
```

```
  select(-c(personid, appyear, schoolsel_chr, major1, major2, minor, major1group, major2group,  
            minorgroup, undergrad_uni))
```

```
#Data Preprocessing
```

Correlation Analysis and Feature Selection

```
data_no <- select(pa.df, c(gpa, stem, schoolsel, essay1length,
                          essay2length, essay3length, essayuniquewords,
                          essaysentiment, signupdate, starteddate,
                          submitteddate, attendedevent))

str(data_no)

data_cor <- as.data.frame(round(cor(data_no), 2))

data_cor
```

Retain relevant predictors by removing highly correlated variables

```
pa.df <- pa.df %>%
  select(-c(essay1length, essay2length, essay3length)) # Retain essayuniquewords
```

Convert categorical variables to factors

```
pa.df <- pa.df %>%
  mutate(across(c(stem, schoolsel, attendedevent, completedadm), as.factor))
```

Scale numeric variables

```
num_vars <- names(select_if(pa.df, is.numeric))
pa.df[num_vars] <- scale(pa.df[num_vars])
```

#Data Partitioning

```
set.seed(1947)

idx <- createDataPartition(pa.df$completedadm, p = 0.8, list = FALSE)

train_data <- pa.df[idx, ]
test_data <- pa.df[-idx, ]

str(pa.df)
```

##KNN##

```
knn_model <- train(
  completedadm ~ ., # Formula specifying the target and predictors
```

```

data = train_data,          # Training data
method = "knn",             # Specify KNN as the model
tuneGrid = data.frame(k = 3:10), # Grid search over k = 3 to k = 10
trControl = trainControl(method = "cv", number = 5) # 5-fold cross-validation
)

```

```

# View model performance during training

```

```

print(knn_model)

```

```

# Make predictions on the test data

```

```

knn_predictions <- predict(knn_model, newdata = test_data)

```

```

# Evaluate the model performance

```

```

confusionMatrix(knn_predictions, test_data$completedadm)

```

```

> confusionMatrix(knn_predictions, test_data$completedadm)
Confusion Matrix and Statistics

```

```

      Reference
Prediction  0    1
      0    98 186
      1 1141 4824

      Accuracy : 0.7876
      95% CI : (0.7773, 0.7977)
      No Information Rate : 0.8017
      P-Value [Acc > NIR] : 0.9973

      Kappa : 0.0591

      Mcnemar's Test P-Value : <2e-16

      Sensitivity : 0.07910
      Specificity : 0.96287
      Pos Pred Value : 0.34507
      Neg Pred Value : 0.80872
      Prevalence : 0.19827
      Detection Rate : 0.01568
      Detection Prevalence : 0.04545
      Balanced Accuracy : 0.52099

```

##Naive Bayes##

```

# Train the Naive Bayes model

```

```

nb_model <- naiveBayes(completedadm ~ ., data = train_data)

```

Make predictions on the test set

```
nb_predictions <- predict(nb_model,newdata= test_data)
```

Evaluate the Naive Bayes model

```
confusionMatrix(nb_predictions, test_data$completedadm)
```

```
> ##Naive_Bayes##
> # Train the Naive Bayes model
> nb_model <- naiveBayes(completedadm ~ ., data = train_data)
> # Make predictions on the test set
> nb_predictions <- predict(nb_model,newdata= test_data)
> # Evaluate the Naive Bayes model
> confusionMatrix(nb_predictions, test_data$completedadm)
Confusion Matrix and Statistics

      Reference
Prediction  0    1
      0    78 170
      1 1161 4840

      Accuracy : 0.787
      95% CI : (0.7766, 0.7971)
      No Information Rate : 0.8017
      P-Value [Acc > NIR] : 0.9982

      Kappa : 0.0415

      McNemar's Test P-Value : <2e-16

      Sensitivity : 0.06295
      Specificity : 0.96607
      Pos Pred Value : 0.31452
      Neg Pred Value : 0.80653
      Prevalence : 0.19827
      Detection Rate : 0.01248
      Detection Prevalence : 0.03969
      Balanced Accuracy : 0.51451

      'Positive' Class : 0
```

#Support Vector Machine (SVM)##

```
svm_model <- ksvm(completedadm ~ ., data = train_data, kernel = "vanilladot")
```

```
svm_pred <- predict(svm_model, test_data)
```

```
confusionMatrix(svm_pred, test_data$completedadm)
```

```

> #Support Vector Machine (SVM)##
> svm_model <- ksvm(completedadm ~ ., data = train_data, kernel = "vanilladot")
  Setting default kernel parameters
> svm_pred <- predict(svm_model, test_data)
> confusionMatrix(svm_pred, test_data$completedadm)
Confusion Matrix and Statistics

          Reference
Prediction 0      1
          0      0
          1 1239 5010

      Accuracy : 0.8017
      95% CI   : (0.7916, 0.8116)
    No Information Rate : 0.8017
    P-Value [Acc > NIR] : 0.5076

      Kappa : 0

McNemar's Test P-Value : <2e-16

      Sensitivity : 0.0000
      Specificity : 1.0000
      Pos Pred Value : NaN
      Neg Pred Value : 0.8017
      Prevalence : 0.1983
      Detection Rate : 0.0000
      Detection Prevalence : 0.0000
      Balanced Accuracy : 0.5000

      'Positive' Class : 0

```

Improving model performance ----

change to a RBF kernel

Train SVM with a linear kernel

```
svm_model_rbf <- ksvm(completedadm ~ ., data = train_data, kernel = "rbfdot")
```

```
svm_pred <- predict(svm_model_rbf, test_data)
```

```
confusionMatrix(svm_pred, test_data$completedadm)
```

```

> # Train SVM with a linear kernel
> svm_model_rbf <- ksvm(completedadm ~ ., data = train_data, kernel = "rbfdot")
> svm_pred <- predict(svm_model_rbf, test_data)
> confusionMatrix(svm_pred, test_data$completedadm)
Confusion Matrix and Statistics

          Reference
Prediction 0      1
          0      7
          1 1232 5010

      Accuracy : 0.8028
      95% CI   : (0.7928, 0.8126)
    No Information Rate : 0.8017
    P-Value [Acc > NIR] : 0.4195

      Kappa : 0.009

McNemar's Test P-Value : <2e-16

      Sensitivity : 0.00565
      Specificity : 1.00000
      Pos Pred Value : 1.00000
      Neg Pred Value : 0.80263
      Prevalence : 0.19827
      Detection Rate : 0.00112
      Detection Prevalence : 0.00112
      Balanced Accuracy : 0.50282

      'Positive' Class : 0

```

##Decision Trees##

```
tree_model <- C5.0(completedadm ~ ., data = train_data)
```

```
tree_pred <- predict(tree_model, test_data)
```

```
confusionMatrix(tree_pred, test_data$completedadm)
```

```
> ##Decision Trees##
> tree_model <- C5.0(completedadm ~ ., data = train_data)
> tree_pred <- predict(tree_model, test_data)
> confusionMatrix(tree_pred, test_data$completedadm)
Confusion Matrix and Statistics

          Reference
Prediction 0      1
0         36     42
1        1203    4968

    Accuracy : 0.8008
    95% CI   : (0.7906, 0.8106)
  No Information Rate : 0.8017
  P-Value [Acc > NIR] : 0.5829

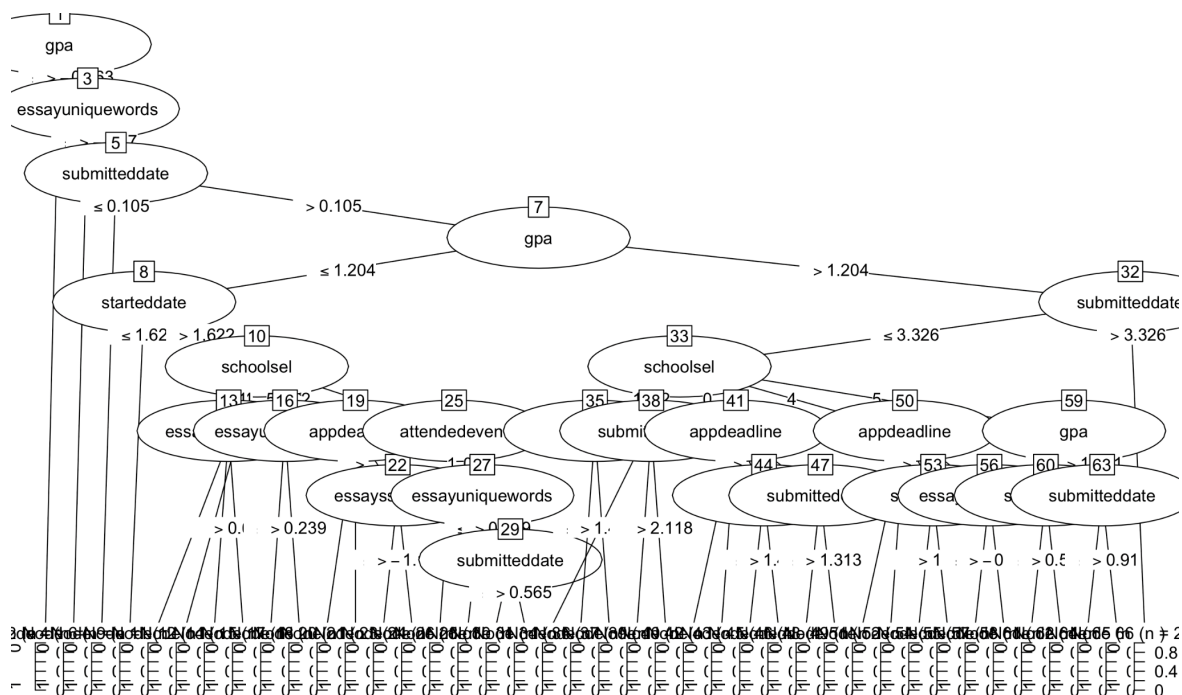
    Kappa : 0.0319

  Mcnemar's Test P-Value : <2e-16

    Sensitivity : 0.029056
    Specificity : 0.991617
   Pos Pred Value : 0.461538
   Neg Pred Value : 0.805056
    Prevalence : 0.198272
    Detection Rate : 0.005761
  Detection Prevalence : 0.012482
   Balanced Accuracy : 0.510336

 'Positive' Class : 0
```

```
plot(tree_model)
```



#TRYING RANDOM FOREST MODEL

set weight

```
class_weights <- c("0" = 10, "1" = 1)
```

train randomforest

```
rf_model <- randomForest(
  completedadm ~ .,
  data = train_data,
  classwt = class_weights)
```

predict & caculate

```
rf_predictions <- predict(rf_model, test_data)
confusionMatrix(rf_predictions, test_data$completedadm)
```

```

> rf_predictions <- predict(rf_model, test_data)
> confusionMatrix(rf_predictions, test_data$completedadm)
Confusion Matrix and Statistics

          Reference
Prediction 0      1
0         66     80
1        1173   4930

      Accuracy : 0.7995
      95% CI : (0.7893, 0.8094)
    No Information Rate : 0.8017
    P-Value [Acc > NIR] : 0.6782

      Kappa : 0.0558

McNemar's Test P-Value : <2e-16

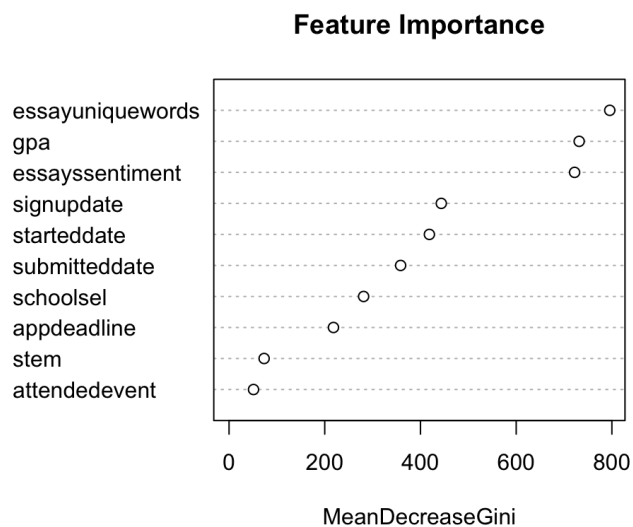
    Sensitivity : 0.05327
    Specificity : 0.98403
   Pos Pred Value : 0.45205
   Neg Pred Value : 0.80780
    Prevalence : 0.19827
    Detection Rate : 0.01056
    Detection Prevalence : 0.02336
    Balanced Accuracy : 0.51865

    'Positive' Class : 0

```

```
importance(rf_model)
```

```
varImpPlot(rf_model, main = "Feature Importance")
```

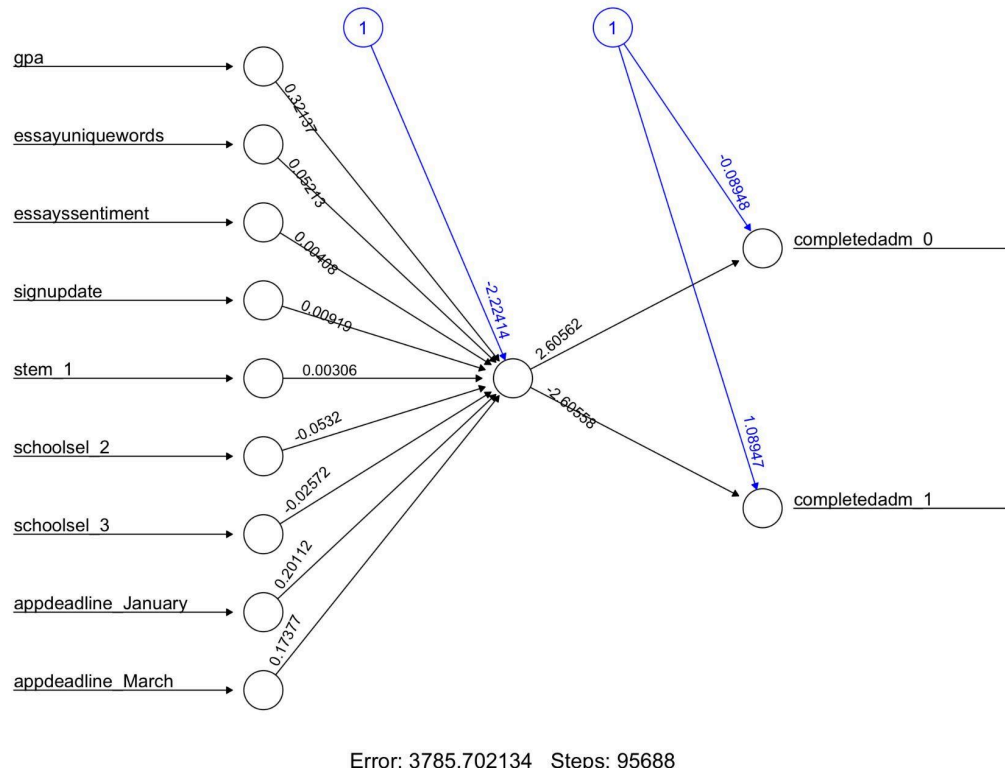


###ANN MODEL##

```
people_cleaned <- pa.df %>%  
  dummy_cols(select_columns = c("stem", "schoolsel", "appdeadline"), remove_first_dummy = TRUE)  
%>%  
  dummy_cols(select_columns = "completedadm", remove_first_dummy = FALSE)  
  
# Split data into training (80%) and testing (20%) set  
set.seed(1947)  
idx <- createDataPartition(people_cleaned$completedadm, p = 0.8, list = FALSE)  
ann_train <- people_cleaned[idx, ]  
ann_test <- people_cleaned[-idx, ]  
  
# Build ANN model with the most important features  
ann_model <- neuralnet(completedadm_0 + completedadm_1 ~  
  gpa + essayuniquewords + essayssentiment + signupdate + stem_1 + schoolsel_2 +  
  schoolsel_3 +  
  appdeadline_January + appdeadline_March,  
  data = ann_train,  
  stepmax = 1e5) # Lower the stepmax for faster convergence
```

```
# Plot ANN
```

```
plot(ann_model)
```



```
# Generate predictions on the test dataset
```

```
ann_predictions <- predict(ann_model, ann_test)
```

```
# Convert probabilities to class labels (0 or 1)
```

```
class.test <- apply(ann_predictions, 1, which.max) - 1 # Subtract 1 to match original labels
```

```
#ConfusionMatrix for ANN
```

```
class.test <- factor(class.test, levels = unique(c(class.test, ann_test$completedadm)))
```

```
ann_test$completedadm <- factor(ann_test$completedadm, levels = unique(c(class.test, ann_test$completedadm)))
```

```
confusionMatrix(class.test, ann_test$completedadm)
```

```
> confusionMatrix(class.test, ann_test$completedadm)
Confusion Matrix and Statistics
```

	Reference	
Prediction	1	0
1	5010	1239
0	0	0

Accuracy : 0.8017
95% CI : (0.7916, 0.8116)
No Information Rate : 0.8017
P-Value [Acc > NIR] : 0.5076

Kappa : 0

Mcnemar's Test P-Value : <2e-16

Sensitivity : 1.0000
Specificity : 0.0000
Pos Pred Value : 0.8017
Neg Pred Value : NaN
Prevalence : 0.8017
Detection Rate : 0.8017
Detection Prevalence : 1.0000
Balanced Accuracy : 0.5000

'Positive' Class : 1